PART 1: SOFTWARE FRAMEWORKS

1.1 Programming vs. Software Engineering

What is Programming?

Programming simply means writing code to create an application. It is the act of translating logic into instructions that a computer can execute.

What is Software Engineering?

Software engineering is the end-to-end process of applying good engineering principles and software development practices to produce quality software.

Key Insight: Software Engineering is NOT just coding. Coding is just one part of a large process.

Other Practices in Software Engineering:

Practice	Description
Requirements Analysis	Analyzing user requirements and designing the software to meet those needs
Stakeholder Communication	Communicating with all stakeholders to resolve issues and provide updates
Architecture & Design	Choosing software architecture, patterns, algorithms, and technologies
Quality Assurance (QA)	Ensuring software meets client requirements for reliability, performance, and security
Deployment	Releasing the software for actual use
Maintenance	Maintaining the software after release as needed

1.2 Generic Functionalities in Software Development

Most web applications share common features:

Common Feature	Description
User Authentication	Login, registration, password management
Routing	Mapping URLs to application resources
Database Connectivity	Connecting to and querying databases
Scheduled Jobs	Automated tasks running at specific times
Session Management	Maintaining user state across requests
Input Validation	Ensuring data meets required formats

Key Question: What if these generic functionalities could be collected and made reusable?

Answer: That's exactly what frameworks provide!

1.3 What is a Framework?

Multiple Definitions:

Source	Definition
Academic	An integrated set of software artifacts (classes, objects, components) that collaborate to provide a reusable architecture for a family of related applications
Practical	A coding environment containing low-level libraries to address conventional coding issues, delivering faster development
Developer	A foundation for developing software applications, used as a template to create websites and applications
Technical	An abstraction providing generic functionality that can be selectively changed by additional user-written code

Key Characteristics of Frameworks

Characteristic	Description
Default Behavior	Before customization, a framework behaves in a manner specific to the user's action
Inversion of Control	The global flow of control is employed by the framework, not the caller
Extensibility	Users can extend the framework by selectively replacing default code
Non-modifiable Framework Code	Users can extend but not modify the core framework code

1.4 How Frameworks Support Software Engineering

Frameworks provide several key elements that support proper software engineering:

Element	Description
Architectural Patterns	MVC, MVVM, etc. provide proven structures
Design Patterns	Reusable solutions to common problems
Inversion of Control	Framework calls your code, not the other way around
Standards & Best Practices	Naming conventions, coding standards, directory structures
Tooling Support	Package managers, dependency managers, boilerplate code generation
Testing Support	Built-in or integrated testing tools

Core Philosophy: "There's no need to reinvent the wheel."

Framework designers aim to facilitate software development by allowing developers to devote time to meeting software requirements rather than dealing with low-level details.

1.5 Framework vs. Libraries

Aspect	Library	Framework
Control	You call the library	The framework calls your code (Inversion of Control)
Scope	Specific functionality	Complete application structure
Extensibility	You extend by adding code	You extend by following framework conventions
Flow	Your code controls flow	Framework controls flow
Example	jQuery, Lodash	Spring, Angular, Django

Inversion of Control Explained:

Library: Your Code → Calls → Library Functions

Framework: Framework → Calls → Your Code (via hooks/extensions)

1.6 Why Use Frameworks?

The main goal: Provide a common structure so developers don't have to redo everything from scratch.

Benefits at a Glance:

Benefit	Description
Code Reusability	Use pre-built, tested components
Standardization	Consistent structure and patterns
Productivity	Faster development once framework is learned
Community	Access to support, plugins, and extensions
Security	Built-in protection against common vulnerabilities

1.7 Advantages of Using Frameworks

Advantage	Description
Improved Coding	Better code organization and structure
Code Reusability	Don't write the same code repeatedly
Ease of Debugging	Easier to find and fix issues compared to from-scratch code
Community Support	Help from other developers, forums, and documentation
Accelerated Development	Faster development once familiar with the framework
Optimized Processes	Caching and optimization for resource-intensive tasks
Less Code	Boilerplate code generation reduces manual coding
Security Standards	Better compliance with accepted security practices

---

1.8 Limitations of Using Frameworks

Limitation	Description
Learning Curve	Learning the framework, not the language, may limit deep language understanding

Constrained by Design	Must respect framework limitations and conventions
Limited Flexibility	Options to tweak functionality are restricted
Feature Bloat	Framework includes many unused features
Performance Impact	Wrong framework choice can negatively impact performance
Version Management	Must keep up with new/deprecated features in each version

Important: Choose the right framework for your application, or performance and user experience may suffer.

1.9 Examples of Popular Frameworks

Backend Frameworks:

Framework	Language
Spring/Spring Boot	Java
Express.js	Node.js/JavaScript
Django	Python
Flask	Python
Laravel	PHP
.NET/.NET Core	C#

Frontend Frameworks:

Framework	Language
Angular	TypeScript
React.js	JavaScript
Vue.js	JavaScript

PART 2: VERSION CONTROLLING WITH GIT- PART I

2.1 What is Version Control?

Definition: "Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later." [1]

Visual Representation:

text

Version 1 → Version 2 → Version 3 → Version 4 → Version 5

↑ ↑ ↑ ↑ ↑

Initial Added Fixed Added Final

Code Feature Bug Feature Version

Common Version Control Methods You May Know:

Method	Description
Multiple Copies	Keeping files with different names (v1, v2, final, final_final)
Cloud Storage	Google Drive, OneDrive with version history
Undo/Redo	Temporary buffer for recent changes

2.2 Manual Version Control: The Amith Scenario

The Scenario:

Amith is working on a file called numbers.py for an assignment over several days.

Amith's Manual System:

1. He saves multiple copies: numbers1.py, numbers2.py, numbers3.py, and numbers.py
2. numbers.py is always the latest version
3. Before making new changes, he renames existing numbers.py to numbersX.py
4. Then saves the new version as numbers.py

The Problem:

At submission time, Amith discovers numbers.py (latest version) throws a serious error.

Question	Answer
Q1: What can Amith do?	Submit the previous version (e.g., numbers10.py)
Q2: What if the latest version has good changes too?	Compare files, copy good parts, paste into previous version
Q3: Concern about HDD failure?	Copy all versions to cloud storage

Analysis of Amith's System:

Pros	Cons
All versions safely stored	Risk of overwriting when syncing from multiple devices
Accessible from anywhere	No high-level overview of changes
Can revert to past versions	Manual process prone to human error
Can compare versions	No automated tracking
Can merge changes	Requires strict discipline

The Realization:

As more people join (like Binesh for pair work), manual systems become unmanageable. A **log file** helps track who did what when, but still requires strict discipline.

Solution: Automated Version Control Systems like Git!

2.3 History of Version Control Systems

Three Generations of VCS:

Generation	Era	Type	Characteristics
First	1973+	Local VCS	Track individual files, one user at a time (e.g., SCCS)
Second	1990s+	Centralized VCS	Central repository, client-server, multiple users (e.g., CVS, SVN)
Third	2000s+	Distributed VCS	Peer-to-peer, every user has full repository (e.g., Git, Mercurial)

First Generation: Local Version Control Systems

- **SCCS (Source Code Control System)** released in 1973 by Bell Labs

- Tracked changes for individual files
- Only one user could edit locally at a time
- Collaboration with large teams was problematic

Second Generation: Centralized Version Control Systems

- Files stored on a centralized repository accessible via network
- Multiple users could use files concurrently
- Follows Client-Server approach
- Requires network access to commit
- Needs a central authority to maintain the repository
- Examples: CVS, Subversion (SVN)

Third Generation: Distributed Version Control Systems

- Emerged with open-source software movement
- No central authority required
- Peer-to-peer (P2P) approach
- Every user maintains a complete local repository
- Examples: Git, Mercurial

2.4 Git: The De-Facto Standard

Git Overview:

Aspect	Details
Creator	Linus Torvalds (creator of Linux)
Year	2005
Pronunciation	"Ghi-T" (not "Ji-T")
Original Purpose	Version control for Linux kernel
Key Feature	Supports non-linear development (thousands of parallel branches)

Important Distinction:

Git ≠ GitHub!

- **Git** is the version control system
- **GitHub** is a hosting service for Git repositories

2.5 Git Basic Concepts

Two Types of Repositories:

Repository Type	Description
Local Repository	Owned by each user on their local machine
Remote Repository	Centralized location accessible to all users (single source of truth)

Why Remote Repositories?

When multiple versions of a project exist, which is the "right" project?
Remote repositories serve as the single source of truth!

Commits:

Concept	Description
Commit	A snapshot of all changes in the local repository (working directory)
Commit ID	Unique SHA-1 hash created for each commit
HEAD	Refers to the currently checked-out branch's latest commit

Basic Git Operations:

text

1. Add a new file to the local repo
2. Make changes to the file
3. Commit the file to the local repo
4. Pull changes from the remote repo
5. Push local changes to the remote repo

2.6 Local vs. Remote Repositories

Obtaining a Local Repository:

Method	Description
Clone	Copy from a remote repository (GitHub, BitBucket, etc.)
Copy	From another user (P2P)
Initialize	Create a brand new repository from scratch

Git Operations Mapped to Amith's Manual System:

Amith's Manual System	Git Equivalent
Revert to past version	git revert
Compare & Merge files	Automatically handled
Push to cloud	git push
Pull from cloud	git pull
Log file	git log, git status, git blame

PART 3: VERSION CONTROLLING WITH GIT- PART II

3.1 Git Branching

What is a Branch?

Analogy: Like a tree branch off the trunk, a Git branch is a detour from the main path of development.

The default branch in Git is master or main (like the trunk of a tree).

Visual Example:

text

```
master:  A---B---C---D---E
       \
feature-x:   C---X---Y---Z
```

3.2 Why Branching Matters

Reason	Description
Parallel Development	Multiple developers work independently (e.g., Amith on Feature A, Binesh on Feature B)
Task Switching	Switch between different tasks easily without interfering code
Experimentation	Try new ideas isolated from stable code
Collaboration	Essential for teams with multiple developers
Isolation	Changes on a branch don't affect the rest of the code

Scenario: Handling Urgent Issues

Without Branching:

1. Working on Feature A in master branch
2. Urgent issue reported
3. Must undo all changes
4. Fix issue

5. Restart work on Feature A

With Branching:

1. Working on Feature A in feature branch
2. Urgent issue reported
3. Switch back to master
4. Create hotfix branch
5. Fix and push
6. Switch back to feature branch
7. Continue where you left off

Key Point: Branching on Git is a very lightweight operation - use it whenever possible!

3.3 Basic Branching Commands

Command	Description
git branch <branch_name>	Create a new branch based on current branch
git checkout <branch_name>	Switch to the specified branch
git checkout -b <branch_name>	Create AND switch to new branch in one command

---

3.4 Git Workflows (Branching Strategies)

A **Git workflow** is a recipe or recommendation for how to use Git to accomplish work in a consistent and productive manner.

Popular Workflows:

Workflow	Complexity	Best For
GitFlow	High	Enterprise, large projects
GitHub Flow	Simple	Most projects, teams
GitLab Flow	Medium	Projects with environments
Trunk-Based Development	Simple	CI/CD, DevOps

3.5 GitFlow Workflow

Characteristics:

- Very comprehensive branching model
- Multiple branch types: master, develop, feature, release, hotfix
- Suited for enterprise-grade software development
- Can become unnecessarily complex for small projects
- Requires enforcement of practices

When to Use:

- Large projects with scheduled releases
- Multiple environments (development, staging, production)
- Teams requiring strict control

Note: For most use cases, GitFlow is overkill.

3.6 GitHub Flow

Six Simple Steps:

text

1. Create a new feature branch from main

↓

2. Make changes to the feature branch

↓

3. Create a Pull Request (PR) for feedback

↓

4. Address review comments

↓

5. Merge PR to main branch

↓

6. Delete feature branch

Key Principles:

- Main branch is always deployable
- Create branches for each feature
- Use Pull Requests for collaboration
- Deploy immediately after merge

Benefits:

- Simple and lightweight
- Works for any project size
- Clear process for collaboration

3.7 Pull Requests (PRs)

What is a Pull Request?

A **Pull Request (PR)** is used to propose a new change to a repository (usually to the main branch).

PR Process:

Step	Description
1. Create	Developer creates PR from feature branch to main
2. Review	Collaborators review the proposed changes
3. Discuss	Feedback and suggestions are shared
4. Update	Developer addresses comments with new commits
5. Approve	Reviewers approve the changes
6. Merge	PR is merged to main branch automatically

Benefits:

- Ensures main branch only contains tested, working code
- Enables code review before integration
- Documents changes and discussions
- Facilitates team collaboration

3.8 Best Practices for Branching

Practice	Description
One Feature Per Branch	Create a branch for each feature; don't work on multiple features in one branch
One Person Per Feature	Ideally, only one person should work on one feature branch
Short-Lived Branches	Feature branches should be deleted once the feature is complete
Delete After Merge	Delete both local and remote feature branches after merging
Merge Early, Merge Often	Avoid merge conflicts by integrating frequently
Keep Main Clean	Only production-ready code should be in master/main
Use Naming Conventions	Stick to a consistent naming pattern (e.g., feature/username/task-name)

Example Naming Convention:

text

feature/vishan.j/awesome-feature-x-to-our-app

feature/amith.k/user-authentication-module

hotfix/security-patch-login

3.9 Git General Best Practices

Practice	Description
Small Incremental Changes	Break big features into smaller, manageable pieces
Atomic Commits	Each commit should focus on one part of the feature
Commit Often	Commit frequently, even if work isn't complete
Pull Before Branching	Pull latest changes from origin/main before creating new branches
Push Frequently	Push local commits to remote often, even for unfinished work

Use Branches	Always use branches, even for individual projects
Write Good Commit Messages	Make messages clear and descriptive
Stick to One Workflow	Choose a branching strategy and use it consistently

Good Commit Message Example:

text

Add user authentication module

- Implement login form with email validation
- Add JWT token generation on successful login
- Create password reset functionality
- Write unit tests for auth controller

Fixes #123

PART 4: WEB APPLICATION ARCHITECTURE

4.1 Website vs. Web Application

Aspect	Website	Web Application
Content	Static content for consumption	Designed for interaction
User Interaction	Read-only primarily	Read-write, user input
Examples	Blog, news site, brochure site	Gmail, Facebook, Online Banking
Focus	Information presentation	Functionality and tasks

Our focus is on Web Applications.

4.2 Three-Tier Architecture

Modern web applications are engineered based on **Three-Tier Architecture**.

The Three Tiers:

text

```
┌──────────────────┐
│ Presentation  │ (Client/Frontend)
│   Tier    │
└──────────┬──────────┘
    │
┌──────────▼──────────┐
│  Application  │ (Server/Backend)
│   Tier    │
└──────────┬──────────┘
    │
┌──────────▼──────────┐
│  Database   │ (Data Storage)
```

```
| Tier |
└──────────────────┘
```

Why Three-Tier Architecture?

Benefit	Description
Independent Infrastructure	Each tier runs on its own infrastructure
Parallel Development	Different teams can develop each tier independently
Independent Scaling	Scale up/down each tier without affecting others
Better Security	Data and logic isolated from presentation tier

---

4.3 Presentation Layer (Frontend)

Also called the **client side** or **frontend**.

Technologies:

Technology	Type
HTML/CSS/JavaScript	Core web technologies
React.js	JavaScript library for UI
Angular	TypeScript-based framework
Vue.js	Progressive JavaScript framework
jQuery + AJAX	Legacy technologies

4.4 Presentation Layer Architectures

Common Architectural Patterns:

Pattern	Description
MVC (Model-View-Controller)	Separates data (Model), UI (View), and logic (Controller)
MVVM (Model-View-ViewModel)	Enhances separation for data binding
MVP (Model-View-Presenter)	Variant for better testability
Component Architecture	Build UI from reusable components

Micro Frontends	Apply microservices concepts to frontend

MVC Pattern Explained:

text

```
┌──────────┐   ┌──────────┐   ┌──────────┐
| Model |◀────┤ Control ├────▶| View |
| (Data) |   | (Logic)|   | (UI) |
└──────────┘   └──────────┘   └──────────┘
  ▲          |
  └──────────────────────────────┘
      Updates
```

4.5 Application Layer (Backend)

Also called the **server side** or **backend**.

Technologies:

Category	Examples
Languages & Frameworks	Java (Spring/Spring Boot), Node.js (Express.js), Python (Django/Flask), PHP (Laravel), .NET
Infrastructure	Virtual Machines (VMs), Serverless Computing, Containers

---

4.6 Application Programming Interfaces (APIs)

What is an API?

API (Application Programming Interface) is a software interface that facilitates communication between two or more applications.

Key Characteristics:

Aspect	Description
Abstraction	Hides complexities of application integrations
Contract	Defines services offered and how to use them

Communication	Enables applications to talk to each other

Benefits of APIs:

Benefit	Description
Productivity	Use existing functions instead of creating from scratch
Cost Reduction	Reduces development efforts significantly
Collaboration	Improves connectivity across the ecosystem

Types of APIs:

Type	Description
REST API	Representational State Transfer (most popular)
SOAP API	Simple Object Access Protocol (legacy)
RPC	Remote Procedure Calls
WebSockets	Real-time bidirectional communication
GraphQL	Query language for APIs

4.7 Application Layer Architectures

Common Architectural Styles:

Architecture	Description
Monolithic	Single unified codebase for all functionality
Microservices	Loosely coupled, independently deployable services
SOA	Service-Oriented Architecture

4.8 Monolithic Architecture

Characteristics:

- Single codebase
- All functionality in one application
- Single deployment unit

Drawbacks:

Drawback	Description
Slower Development	Local complexity slows down progress
Scaling Issues	Can't scale individual components
Reliability	Error in one module can affect entire application
Technology Barrier	Changes in framework/language affect whole app
Lack of Flexibility	Constrained by existing technologies
Redeployment	Small change requires full redeployment

4.9 Microservices Architecture

Definition:

"Microservices architecture is an approach in which a single application is composed of many loosely coupled and independently deployable smaller services." - IBM

Characteristics:

Feature	Description
Small Services	Each service handles specific functionality
Independent	Services are loosely coupled
Independent Deployment	Update services without rebuilding entire app
Independent Scaling	Scale each service based on demand
Polyglot Programming	Use different languages for different services

Benefits:

Benefit	Description
Team Autonomy	Single team can own a service
Independent Deployment	Update services individually
Scalability	Scale only what needs scaling
High Reliability	Failures isolated to specific services
Technology Flexibility	Choose best tech for each service

Challenges:

Challenge	Description
Global Complexity	Interactions between services create complexity
Infrastructure Costs	More services = more infrastructure
Debugging Difficulty	Tracing issues across services is hard
Network Latency	Service communication adds overhead
Data Management	Maintaining data consistency across services

Visual Example - Uber's Architecture:

Uber's microservice architecture circa mid-2018 had hundreds of services, creating significant global complexity.

Key Insight: Local complexity depends on service implementation; global complexity is defined by interactions and dependencies between services.

4.10 Other Helpful Technologies

Technology	Purpose
API Gateways	Authentication, throttling, orchestration, caching
CDN (Content Delivery Networks)	Distribute content globally for faster access
Caching Tools	Redis, Memcached for performance
Load Balancers	Distribute traffic across servers
Message Queues	Asynchronous communication between services
Cloud Computing	On-demand computing resources
Cloud Storage	Scalable data storage
Virtual Machines	Isolated computing environments

Advanced Concepts:

- **BFF (Backend For Frontend)** - Backend specifically designed for a frontend
- **Micro Frontends** - Apply microservices concepts to frontend development

---

PART 5: REST APIS

5.1 Consuming Web Resources

Human Consumption:

- Web browsers
- Web applications
- Mobile applications

Program Consumption:

- **Web APIs** allow programs to consume web resources

5.2 What is an API?

Definition Recap:

A software interface that facilitates communication between two or more applications.

API Examples (Historical to Modern):

Technology	Status
CORBA	Legacy
SOAP	Legacy
RPC	Legacy
REST	Modern (Most popular)
GraphQL	Modern

5.3 What is REST?

REST Overview:

Aspect	Details
Full Name	Representational State Transfer
Creator	Roy Fielding

Year	2000 (Doctoral dissertation)
Type	Architectural style (not a protocol)
Status	Most popular approach for Web APIs

Why REST is Popular:

Reason	Description
Architectural Style	Not a technology, just guidelines
Flexibility	No strict rules; aligns with constraints = REST API
Simplicity	Easier than SOAP/CORBA
HTTP-Based	Runs on familiar HTTP protocol

5.4 Six Architectural Constraints of REST

1. Client-Server Architecture

Requirement	Description
Clear Separation	Maintain separation between client and server
Purpose	Promote portability of UI
Purpose	Promote low coupling
Purpose	Improve scalability of server components
Purpose	Allow independent growth

2. Stateless

Requirement	Description
No Server Context	Each request must contain all information needed
Session State	Session state kept entirely on client
Benefits	Improves visibility, reliability, and scalability

3. Cacheable

Requirement	Description
Label Responses	Responses labeled as cacheable or non-cacheable

Client Cache	Clients can reuse cacheable responses
Benefits	Eliminates some interactions, improves efficiency and scalability

4. Uniform Interface (Four Sub-Constraints)

Sub-Constraint	Description
Self-descriptive Messages	Each message contains enough information to process it
Identification of Resources	Resources identified via URIs (separate from representations)
Resource Manipulation	Client can modify/delete with representation + metadata
HATEOAS	Hypermedia as the Engine of Application State

HATEOAS Explained (Hey-tee-oh-as):

Having accessed an initial URI, a REST client should use server-provided links to dynamically discover all available resources.

Analogy: Like a human user knowing only the main URL of a website; the rest is discovered while navigating.

Example:

text

```
{
 "id": 1,
 "name": "John Doe",
 "email": "john@example.com",
 "_links": {
  "self": { "href": "/users/1" },
  "orders": { "href": "/users/1/orders" },
  "update": { "href": "/users/1" },
  "delete": { "href": "/users/1" }
 }
}
```

5. Layered System

Requirement	Description
Hierarchical Layers	Components can't see beyond immediate layer
Intermediaries	Proxies, gateways can be transparently deployed
Common Uses	Security enforcement, response caching, load balancing

6. Code on Demand (Optional)

Requirement	Description
Extend Client	Download and execute code (applets, scripts)
Benefit	Simplifies clients by reducing pre-implemented features
Status	Only optional constraint in REST

5.5 REST vs. RPC

Common Misconceptions:

Not REST	Why?
Pretty URLs like /employees/10	Just one aspect, not sufficient
Using HTTP methods (GET, POST)	Can be RPC with HTTP
Having all CRUD operations	Missing HATEOAS

RPC-Style API (Missing HATEOAS):

```
json
{
 "id": 1,
 "name": "John Doe",
 "email": "john@example.com"
}
```

RESTful API (With HATEOAS):

```
json
{
 "id": 1,
```

```
  "name": "John Doe",
  "email": "john@example.com",
  "_links": {
    "self": { "href": "/employees/1" },
    "department": { "href": "/employees/1/department" },
    "manager": { "href": "/employees/1/manager" },
    "subordinates": { "href": "/employees/1/subordinates" }
  }
}
```

5.6 Richardson Maturity Model

A four-level scale indicating how much an API conforms to REST constraints.

The Four Levels:

text

Level 3: HATEOAS (Full REST)

↑

Level 2: HTTP Verbs (Methods)

↑

Level 1: Resource-Based URIs

↑

Level 0: Swamp of POX

Level	Name	Description
Level 0	Swamp of POX	One URI, all actions via POST (SOAP, XML-RPC)
Level 1	Resource-Based	Individual URIs for resources, still all POST
Level 2	HTTP Verbs	Full use of GET, POST, PUT, DELETE

Level 3	HATEOAS	URIs + HTTP Methods + HATEOAS

POX = Plain Old XML (could be JSON too)

5.7 Facilitators of REST

HTTP Methods:

Method	Operation
GET	Reading a resource
POST	Writing/creating a resource
PUT	Updating a resource
DELETE	Deleting a resource

HTTP Status Codes:

Range	Category	Example
2xx	Success	200 OK
4xx	Client Error	404 Not Found
5xx	Server Error	500 Internal Server Error

HTTP Headers:

Header	Purpose
Content-Type	Format of data being sent
Cache-Control	Caching directives
Authorization	Authentication credentials

Data Formats:

Format	Description
JSON	JavaScript Object Notation - lightweight, human-readable key-value pairs
HAL	Hypertext Application Language - structure for RESTful resources with links

---

5.8 REST Best Practices

Practice	Description
Use Correct HTTP Methods	GET for read, POST for create, PUT for update, DELETE for delete
Use Proper Status Codes	2xx for success, 4xx for client errors, 5xx for server errors
Use HTTP Headers	Content-Type, Cache-Control, Authorization appropriately
Accept/Respond with JSON	JSON is the standard for modern REST APIs
Use Authentication	OAuth, JWT, etc. for security
Meaningful URIs	/users/, /users/2, /orders/pending
Keep Consistent	Follow same patterns throughout API
Include HATEOAS	Aim for Level 3 maturity

Good URI Examples:

text

https://api.example.com/users/

https://api.example.com/users/2

https://api.example.com/users/2/orders

https://api.example.com/products?category=electronics

PART 6: QUICK REFERENCE TABLES

6.1 Key Terms and Definitions

Term	Definition
Framework	Integrated set of software artifacts providing reusable architecture
Software Engineering	End-to-end process of applying engineering principles to produce quality software
Programming	Writing code to create an application
Version Control	System that records changes to files over time
Git	Distributed version control system created by Linus Torvalds
Commit	Snapshot of all changes in a repository
Branch	Detour from main development path
Pull Request	Proposal to merge changes from one branch to another
API	Interface facilitating communication between applications
REST	Architectural style for creating web services
HATEOAS	Hypermedia as the Engine of Application State

6.2 Git Commands Cheat Sheet

Command	Description
git init	Initialize a new repository
git clone <url>	Copy a remote repository locally
git add <file>	Stage changes for commit
git commit -m "message"	Commit staged changes
git status	Check status of working directory
git log	View commit history
git branch <name>	Create a new branch

git checkout <branch>	Switch to a branch
git checkout -b <branch>	Create and switch to new branch
git merge <branch>	Merge branch into current branch
git pull	Fetch and merge from remote
git push	Push local commits to remote
git revert <commit>	Revert a commit
git blame <file>	See who changed what in a file

6.3 HTTP Methods and Status Codes

HTTP Methods:

Method	CRUD Equivalent	Description
GET	Read	Retrieve a resource
POST	Create	Create a new resource
PUT	Update	Update an existing resource
DELETE	Delete	Remove a resource

Common HTTP Status Codes:

Code	Meaning	Category
200	OK	Success
201	Created	Success
204	No Content	Success
400	Bad Request	Client Error
401	Unauthorized	Client Error
403	Forbidden	Client Error
404	Not Found	Client Error
500	Internal Server Error	Server Error
503	Service Unavailable	Server Error

6.4 REST Constraints Summary

Constraint	Required?	Description
Client-Server	Yes	Separate client from server
Stateless	Yes	Each request contains all information
Cacheable	Yes	Responses labeled cacheable/non-cacheable
Uniform Interface	Yes	Four sub-constraints (including HATEOAS)
Layered System	Yes	Hierarchical layers with intermediaries
Code on Demand	No	Download and execute code

Uniform Interface Sub-Constraints:

1. **Self-descriptive Messages**
2. **Identification of Resources** (URIs)
3. **Resource Manipulation through Representations**
4. **HATEOAS** (Hypermedia as Engine of Application State)

6.5 Architecture Comparison

Monolithic vs. Microservices:

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent services
Scaling	Scale entire app	Scale individual services
Development	Single codebase	Multiple codebases
Complexity	Local complexity high	Global complexity high
Team Structure	Large team	Small, focused teams
Technology	Single tech stack	Polyglot
Reliability	One failure affects all	Failures isolated

Git Workflows Comparison:

Aspect	GitFlow	GitHub Flow

Complexity	High	Low
Branches	Multiple (master, develop, feature, release, hotfix)	Main + feature branches
Best For	Enterprise, scheduled releases	Continuous deployment, most projects
Learning Curve	Steep	Gentle